

Chapter 7: Deadlocks

Chien Chin Chen

Department of Information Management
National Taiwan University

Outline

- System Model.
- Deadlock Characterization.
- Methods for Handling Deadlocks.
- Deadlock Prevention.
- Deadlock Avoidance.
- Deadlock Detection.
- Recovery from Deadlock.

Chapter Objectives

- To develop a description of deadlocks, which prevent sets of concurrent processes from completing their tasks.
- To present a number of different methods for preventing or avoiding deadlocks in a computer system.

System Model (1/6)

- A computer system consists of a finite number of **resources**.
 - The resources are partitioned into several **resource types**.
 - Can be either **physical** resources.
 - E.g., CPU, printers ...
 - Or **logical** resources.
 - E.g., files.
 - Each consisting of some number of **identical instances**.
 - E.g., systems with two CPUs.
 - A system has two printers; one is on the ninth floor and the other is in the basement.
 - The printers **may not be identical** because people on the ninth floor may not print document in the basement.
 - Instances are defined to be in the same resource type **if no one cares which instance carry out the task.**

System Model (2/6)

- Processes may **compete** for a finite number of resources (instances) to fulfill their tasks.
 - A process utilizes a resource in the following sequence:
 - **Request**: If the request cannot be granted immediately (the resource is being used by other processes), then the requesting process must wait until it can acquire the resource.
 - **Use**: The process can operate on the resource (e.g., print on the printer).
 - **Release**: The process releases the resource.
- The request and release of resources are usually system calls.
 - E.g., `open()` and `close()` file, or `allocate()` and `free()` memory.
 - For kernel-managed resources, the operating system maintains a system table which records:
 - Whether each resource is free or allocated.
 - The process to which a resource is allocated.
 - A queue of processes waiting for a resource.

System Model (3/6)

- (rarely) Request and release of resources that are not managed by the operating system can be accomplished through:
 - `wait()` and `signal()` operations on counting semaphores or mutex locks.

System Model (4/6)

- A **set** of processes is in a **deadlock** state when ...
 - Every process in the set is **waiting for an event** that can be caused only by another process in the set.
 - Event – resource acquisition and release.
- Example of deadlocks involved different resource types:
 - A system with one printer and one DVD drive.
 - Process P_i is holding the DVD.
 - Process P_j is holding the printer.
 - If P_i requests the printer and P_j requests the DVD drive, a deadlock occurs.

System Model (5/6)

- *Multithreaded programs* are good candidates for deadlock because multiple threads can compete for shared resources.
 - For example, two Pthread mutex locks are created and used in the following multithreaded Pthread programs.

```
/* create and initialize mutex locks */
pthread_mutex_t first_mutex;
pthread_mutex_t second_mutex;

pthread_mutex_init(&first_mutex, NULL);
pthread_mutex_init(&second_mutex, NULL);
```

```
void *do_work_one (void *param)
{
    pthread_mutex_lock(&first_mutex);
    pthread_mutex_lock(&second_mutex);
    /* do some work */
    pthread_mutex_unlock(&second_mutex);
    pthread_mutex_unlock(&first_mutex);
    pthread_exit(0);
}
```

```
void *do_work_two (void *param)
{
    pthread_mutex_lock(&second_mutex);
    pthread_mutex_lock(&first_mutex);
    /* do some work */
    pthread_mutex_unlock(&first_mutex);
    pthread_mutex_unlock(&second_mutex);
    pthread_exit(0);
}
```

System Model (6/6)

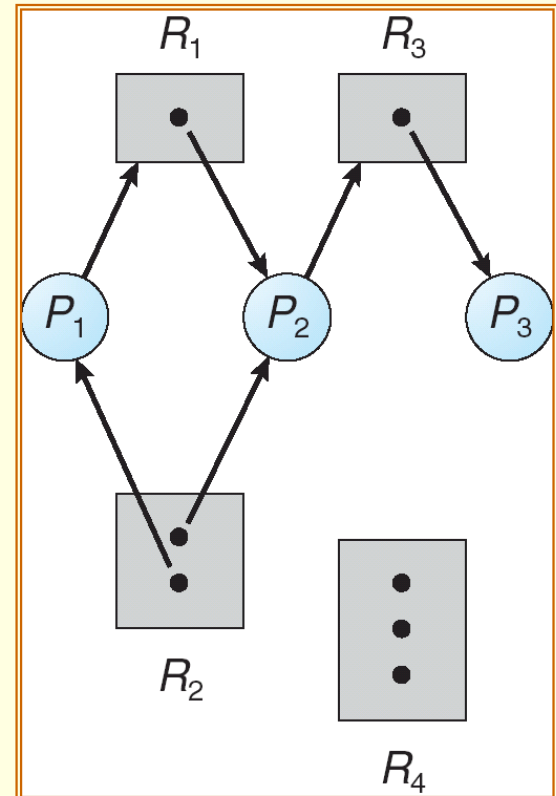
- `thread_one` (`do_work_one`) attempts to acquire the mutex locks in the order (1) `first_mutex`, (2) `second_mutex`.
- `thread_two` (`do_work_two`) attempts to acquire the mutex locks in the order (1) `second_mutex`, (2) `first_mutex`.
- Deadlock is **possible** if `thread_one` acquires `first_mutex` while `thread_two` acquires `second_mutex`.
 - But **will not occur** if `thread_one` is able to acquire and release the mutex locks **before** `thread_two` attempts to acquire the lock.
- Obviously, deadlock handling is a tough task:
 - It is difficult to identify and test for deadlocks that may occur only under certain circumstances.

Deadlock Characterization

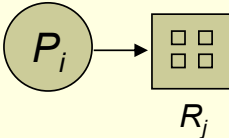
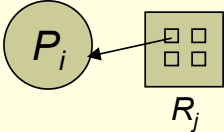
- Deadlock can arise if four conditions **hold simultaneously**.
 - **Mutual exclusion**: only one process at a time can use a resource.
 - **Hold and wait**: a process holding at least one resource is waiting to acquire additional resources held by other processes.
 - **Circular wait**: there exists a set $\{P_0, P_1, \dots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1 , P_1 is waiting for a resource that is held by P_2 , ..., P_{n-1} is waiting for a resource that is held by P_n , and P_n is waiting for a resource that is held by P_0 .
 - **No preemption**: a resource can be released only voluntarily by the process holding it, after that process has completed its task.

Resource-Allocation Graph (1/5)

- Deadlocks can be described more precisely in terms of a **directed** graph – **system resource-allocation graph**.
- The graph consists of a set of vertices **V** and a set of edges **E**.
 - **V** is partitioned into two types:
 - $P = \{P_1, P_2, \dots, P_n\}$, the set consisting of all the **processes** in the system.
 - $R = \{R_1, R_2, \dots, R_m\}$, the set consisting of all **resource types** in the system.
 - Two types of directed edges **E**:
 - **Request edge** – directed edge $P_i \rightarrow R_j$
 - **Assignment edge** – directed edge $R_j \rightarrow P_i$

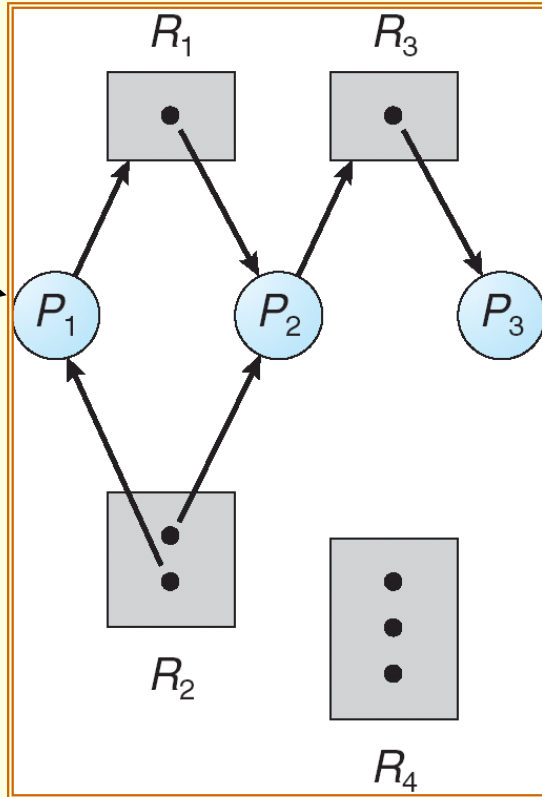


Resource-Allocation Graph (2/5)

- Process: 
- Resource Type with 4 **instances**: 
- Request edge: P_i requests instance of R_j : 
- Assignment edge: P_i is holding an instance of R_j : 
- When a request can be fulfilled, the request edge is instantaneously transformed to an assignment edge.
- When the process no longer needs access to the resource, it releases the resource and the assignment edge is deleted.

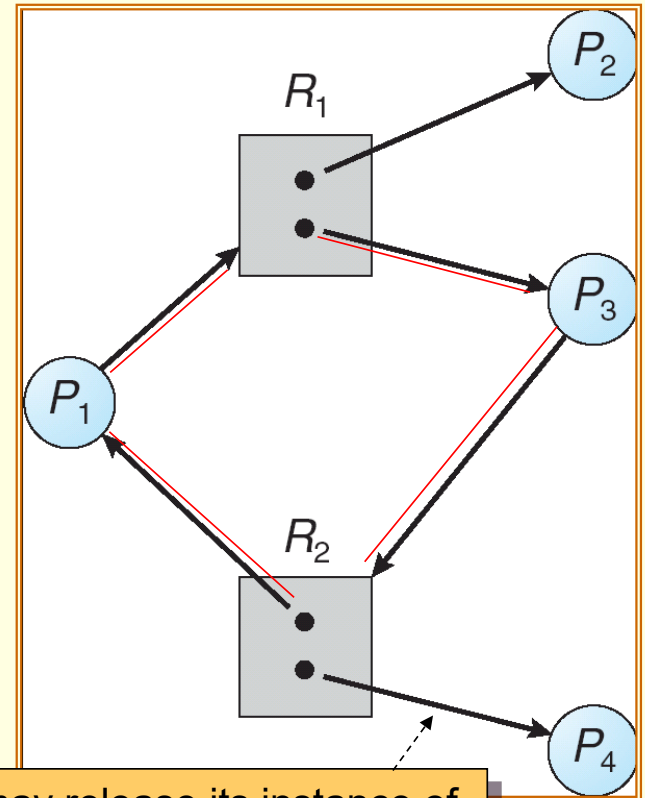
Resource-Allocation Graph (3/5)

Process P_1 is holding an instance of resource type R_2 and is waiting for an instance of resource type R_1 .



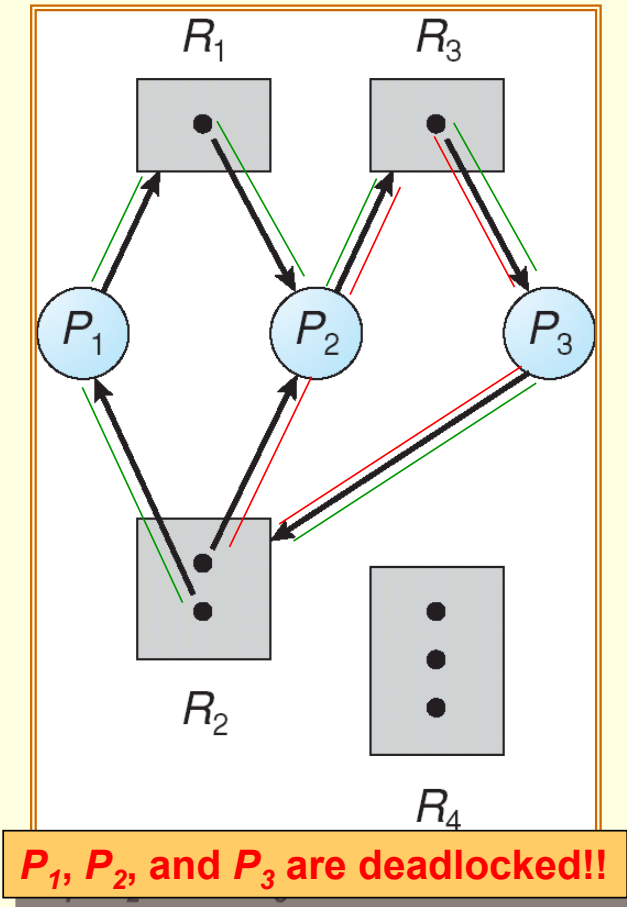
Resource-Allocation Graph (4/5)

- If each resource type has **exactly one instance**, then a cycle of a graph implies that a deadlock has occurred.
 - Each process involved in the cycle is deadlocked.
- If each resource type **has several instances**, then a cycle **does not necessarily** imply that a deadlock has occurred.



Process P_4 may release its instance of resource type R_2 , breaking the cycle.

Resource-Allocation Graph (5/5)



■ Guideline:

- If a resource-allocation graph does not have a cycle, then the system is not in a deadlocked state.
- If there is a cycle, then the system **may or may not be** in a deadlocked state.
 - If only one instance per resource type, then deadlock.
 - If several instances per resource type, possibility of deadlock.

Methods for Handling Deadlocks (1/4)

- We can deal with the deadlock problem in one of three ways:
 - Design a protocol to **prevent** or **avoid** deadlocks, ensure that the system will *never* enter a deadlock state.
 - Allow the system to enter a deadlock state, **detect** it and **recover**.
 - Ignore the problem and pretend that deadlocks never occur in the system;
 - Used by most operating systems, including UNIX and Windows.
 - It is then up to the application developer to write programs that handle deadlocks. (very difficult)
- Researchers argued that none of the basic approaches alone is appropriate for the deadlock problem.
 - An optimal solution is to combined the basic approaches.

Methods for Handling Deadlocks (2/4)

■ *Deadlock prevention:*

- Methods for ensuring that **at least one** of the necessary conditions cannot hold.
- Prevent deadlocks by constraining how requests for resources can be made.

■ *Deadlock avoidance:*

- Require that the operating system be given **in advance additional information** concerning which resources a process will request and use during its lifetime.
- With this additional knowledge, the system can consider the resources currently available, the resource currently allocated, and the future requests and releases of each process ... to avoid deadlocks.

Methods for Handling Deadlocks (3/4)

- If a system does not employ either a deadlock-prevention or a deadlock-avoidance algorithm ...
 - A deadlock situation may arise!!
- The system can provide an algorithm to **detect** deadlocks, and an algorithm to **recover** from the deadlocks.
- If no deadlock prevention, deadlock avoidance, or deadlock-detection-recovery ...
 - The system **may** arrive at a deadlocked state yet **has no way of recognizing what has happened**.
 - More and more processes will enter a deadlocked state.
 - Decrease the system's performance.
 - Eventually, the system will need to be restarted manually.

Methods for Handling Deadlocks (4/4)

- This method (manual re-start) is used in most operating systems.
 - Deadlock occur infrequently (probably once per year!!).
 - This method is thus cheaper than other approaches, which must be used constantly.

Deadlock Prevention (1/9)

- By ensuring that **at least one** of four necessary conditions cannot hold, we can prevent the occurrence of a deadlock.
- **Mutual Exclusion:**
 - In general, we **cannot** prevent deadlocks by denying the mutual-exclusion condition.
 - You definitely do not want to share a printer with others when using it.
 - Sharable resources do not require mutually exclusive access and thus cannot be involved in a deadlock.
 - For example, read-only files.

Deadlock Prevention (2/9)

■ Hold and Wait:

- We must guarantee that whenever a process requests a resource, it does not hold any other resources.
- Consider a process that copies data from DVD drive to a file on disk, sorts the file, and then prints the results to a printer.
- **Protocol one:**
 - Require process to request and be allocated all its resources before it begins execution.
 - The process must initially request the DVD drive, disk file, and printer.
 - The printer will be held for the entire execution, even though it is used at the end!!

Deadlock Prevention (3/9)

■ Protocol two:

- Allow process to request resources only when the process has none.
- A process may request some resources and use them.
- Before it can request any additional resources, it must release all the resources that it is currently allocated.
- The process first requests only the DVD drive and disk file.
 - It copies from the DVD drive to the disk.
- It then releases both the DVD drive and the disk file. (none)
- And again request the disk file and the printer.
 - Copying the disk file to the printer.
- Finally, it releases these two resources and terminates.

Deadlock Prevention (4/9)

- Disadvantages of the methods:
 - **Low resource utilization:**
 - Resources may be allocated but unused for a long period.
 - If we cannot be assured that our data will remain on the disk file (modified by other process when we release the disk file and DVD drive), then we must request all resources at the beginning for both protocols.
 - **Starvation is possible:**
 - A process that needs **several popular** resources may have to wait indefinitely.
 - At least one of the resources that it needs is always allocated to some other process.

Deadlock Prevention (5/9)

■ No Preemption:

- Is often applied to resources whose state can be easily saved and restored later (such as memory space).
- It cannot generally be applied to such resources as printers and tape drives.

■ Protocol one:

- If a process that is holding some resources and requests another resource that cannot be immediately allocated to it, then all resources currently being held are preempted.
- Preempted resources are added to the list of resources for which the process is waiting.
- Process will be restarted only when it can regain its old resources, as well as the new ones that it is requesting.

Deadlock Prevention (6/9)

■ Protocol two:

- If a process requests some resources, we first check whether they are available.
- If they are, we allocate them.
- If not, we check whether they are allocated to some other process that is waiting for additional resources.
- If so, we preempt the desired resources from the waiting process and allocate them to the requesting process.
- If the resources are neither available nor held by a waiting process (used by running processes), the requesting process must wait.
 - When waiting, its resources may be preempted if another process requests them.

Deadlock Prevention (7/9)

■ Circular Wait:

- Impose a **total ordering** of all resource types.
- Require that each process requests resources in an increasing order of enumeration.
- We can define a one-to-one function $F: R \rightarrow N$.
 - N is the set of natural number, $\{1, 2, 3, \dots\}$.
 - $R = \{R_1, R_2, \dots, R_m\}$ is the set of resource types.
 - For example, R includes tape drives, disk drives, and printer; the function F might be defined as follows:

$F(\text{tape drive}) = 1$

$F(\text{disk drive}) = 5$

$F(\text{printer}) = 12$

Deadlock Prevention (8/9)

■ Protocol:

- A process can initially request any number of instances of a resource type – say R_i .
- After that, the process can request instances of resource type R_j if and only if $F(R_j) > F(R_i)$.

So, there can be no circular wait!!

■ Proof (circular-wait condition cannot hold):

- Let the set of process processes involved in the circular wait be $\{P_0, P_1, \dots, P_n\}$.
 - P_i is waiting for a resource R_i , held by process P_{i+1} .
 - P_n is waiting for a resource R_n held by P_0 .
- Since P_{i+1} is holding R_i while requesting R_{i+1} , we have $F(R_i) < F(R_{i+1})$, for all i .
- Then, $F(R_0) < F(R_1) < \dots < F(R_n) < F(R_0)$

$F(R_0) < F(R_0) \dots$ impossible

Deadlock Prevention (9/9)

- We can develop an ordering among all synchronization objects in the system.
 - All requests for the objects must be made in increasing order.
 - For example:
$$F(\text{first_mutex}) = 1$$
$$F(\text{second_mutex}) = 5$$
- Note that, the function F should be defined according to the normal order of usage of the resources in a system.
 - Usually, tape drive is needed before the printer, so it would be reasonable to define $F(\text{tape drive}) < F(\text{printer})$.

Deadlock Avoidance (1/6)

- Possible side effects of preventing deadlocks are low resource utilization and reduced system throughput.
- Given **additional information**, it is possible to construct an algorithm that ensures that the system will never enter a deadlocked state.
 - Example of additional information - the maximum number of resource of each type that a process may need.
- A deadlock-avoidance algorithm dynamically examines the **resource-allocation state** to ensure that a circular-wait condition can never exist.
 - **Resource-allocation state** – the number of available and allocated resources and the maximum demands of the processes.

Deadlock Avoidance (2/6)

- A state is **safe** if the system can allocate resources to each process in some order and still avoid a deadlock.
 - There exists a **safe sequence** of processes $\langle P_1, P_2, \dots, P_n \rangle$ such that:
 - For each P_i , the resource requests that P_i can still make can be satisfied by the currently available resources plus the resources held by all P_j , with $j < i$.
 - If the resources that P_i needs are not immediately available, P_i can wait until all P_j have finished.
 - Then, P_i can complete its task, return its allocated resource and terminate.
 - P_{i+1} then can obtain its needed resources, and so on.
- If no such sequence exists, then the system is said to be **unsafe**.

Deadlock Avoidance (3/6)

- Example: a system with 12 magnetic tape drivers and three processes: P_0 , P_1 , and P_2 .

- At time t_0 :

additional information

	Max Needs	Current Needs (hold)
P_0	10	5
P_1	4	2
P_2	9	2

available:
3

- Is the system in a safe state? **YES!!**
- The sequence $\langle P_1, P_0, P_2 \rangle$ satisfies the safety condition.

deadlock $\neq$ unsafe

Deadlock Avoidance (4/6)

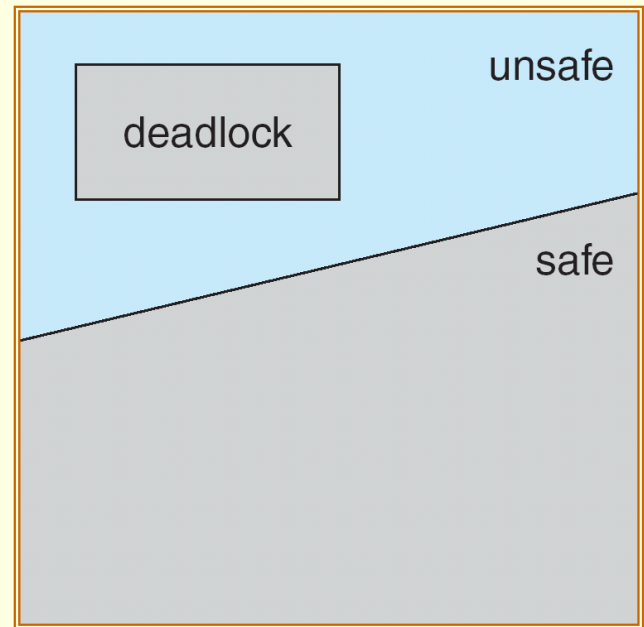
- A system can go from a safe state to an unsafe state:
 - At time t_0 :

	Max Needs	Current Needs (hold)	available: 3
P_0	10	5	
P_1	4	2	
P_2	9	2	

- At time t_1 , process P_2 requests and is allocated one more tape drive.
 - Can we find a safe sequence?
 - ... we cannot find a safe sequence.
 - Our mistake was in granting the request from P_2 for one more tape drive.

Deadlock Avoidance (5/6)

- A safe state is not a deadlocked state.
- A deadlocked state is an unsafe state.
- Not all unsafe states are deadlocks!!
 - An unsafe state **may** lead to a deadlock.



Deadlock Avoidance (6/6)

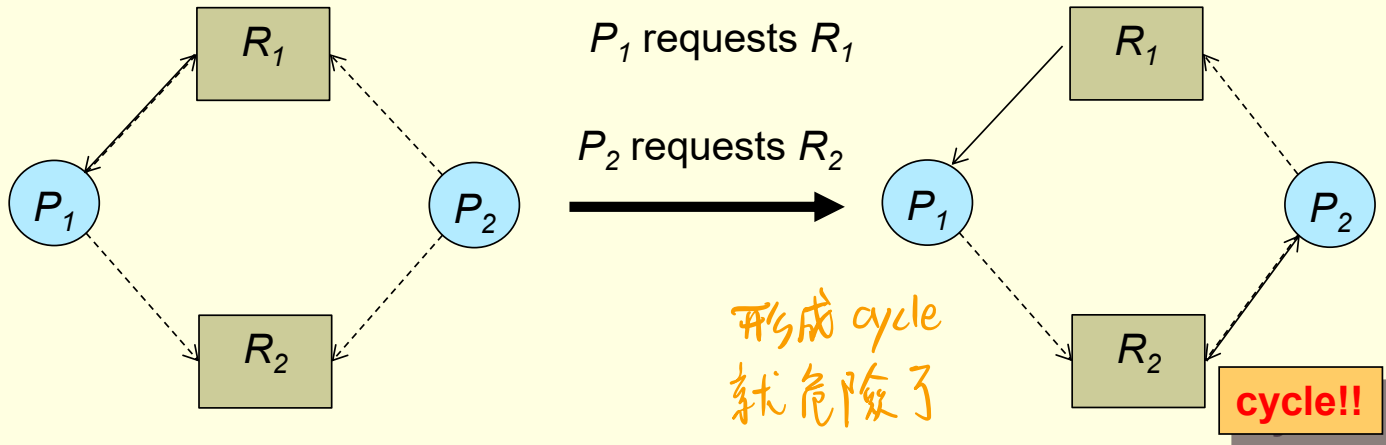
- Avoidance algorithms ensure that the system will always remain in a safe state.
 - Initially, the system is in a safe state.
 - Whenever a process requests a resource that is currently available, the system must decide whether the resource can be allocated or whether the process must wait.
 - The request is granted only if the allocation leave the system in a safe state.

Resource-Allocation-Graph Algorithm (1/2)

- For one instance of each resource type only.
- In addition to the request and assignment edges, we introduce a new type of edge, called a **claim edge**.
 - A claim edge $P_i \rightarrow R_j$ indicates that process P_i may request resource R_j at some time in the future.
 - The edge resembles a request edge in direction but is represented by a **dashed line**.
- Before process P_i starts executing, all its claim edges must already appear in the resource-allocation graph.
 - The a priori information.
- When P_i requests R_j , the claim edge $P_i \rightarrow R_j$ is converted to a request edge.
 - When R_j is released by P_i , the assignment edge $R_j \rightarrow P_i$ is reconverted to a claim edge $P_i \rightarrow R_j$.

Resource-Allocation-Graph Algorithm (2/2)

- When process P_i requests resource R_j ,
 - The request can be granted only if the edge conversion **does not** result in the formation of a cycle in the graph.



- An graph algorithm for detecting a cycle require an order of n^2 , where n is the number of processes in the system.

Banker's Algorithm (1/9)

- For multiple instances of each resource type.
- When a process enters the system, it must declare the maximum number of instances of each resource type that it may need.
- The system must determine whether the allocation of the resources (of a process request) will leave the system in a safe state.

Banker's Algorithm (2/9)

■ Data structures:

- Let n = number of processes, and m = number of resources types.

■ *Available*:

- vector of length m .
- If $Available[j] = k$, there are k instances of resource type R_j available.

■ *Max*:

- $n \times m$ matrix.
- If $Max[i, j] = k$, then process P_i may request at most k instances of resource type R_j

■ *Allocation*:

- $n \times m$ matrix.
- If $Allocation[i, j] = k$ then P_i is currently allocated k instances of R_j .

■ *Need*:

- $n \times m$ matrix.
- If $Need[i, j] = k$, then P_i may need k more instances of R_j to complete its task.

$$Need[i, j] = Max[i, j] - Allocation[i, j].$$

These data structures vary over time in both size and value.

Banker's Algorithm (3/9)

■ Notation:

- Let X and Y be vectors of length n .
- $X \leq Y$ if and only if $X[i] \leq Y[i]$ for all $i = 1, 2, \dots, n$.
- Example:

$$X = \begin{array}{|c|} \hline 1 \\ \hline 7 \\ \hline 3 \\ \hline 2 \\ \hline \end{array} \quad Y = \begin{array}{|c|} \hline 0 \\ \hline 3 \\ \hline 2 \\ \hline 1 \\ \hline \end{array} \quad \text{then } Y \leq X.$$

- $Y < X$ if $Y \leq X$ and $Y \neq X$.

Banker's Algorithm (4/9)

■ Safety algorithm:

- Find out whether or not a system is in a safe state.

1. Let *Work* and *Finish* be vectors of length *m* and *n*, respectively.

Initialize:

Work = *Available*

Finish[*i*] = *false* for *i* = 0, 1, 2, ..., *n*-1.

用 *work* 代表水位高低
以免影响到 *available*

2. Find an *i* such that both:

(a) *Finish*[*i*] = *false*

(b) $Need_i \leq Work$

If no such *i* exists, go to step 4.

Row *i* in the matrix *Need*.

Banker's Algorithm (5/9)

3. $Work = Work + Allocation_i$
 $Finish[i] = true$
go to step 2.

Row i in the matrix $Allocation$.

4. If $Finish[i] == true$ for all i , then the system is in a safe state.

Banker's Algorithm (6/9)

■ Resource-request algorithm:

- Determine if requests can be safely granted.
- Let $Request_i$ be the request vector for process P_i .
 - If $Request_i[j] = k$ then process P_i wants k instances of resource type R_j .
- 1. If $Request_i \leq Need_i$ go to step 2. (能滿足最大需求量)
Otherwise, raise error condition, since process has exceeded its maximum claim.
- 2. If $Request_i \leq Available$, go to step 3. Otherwise P_i must wait, since resources are not available.

Banker's Algorithm (7/9)

3. **Pretend** to allocate requested resources to P_i by modifying the state as follows:

$$Available = Available - Request_i;$$

$$Allocation_i = Allocation_i + Request_i;$$

$$Need_i = Need_i - Request_i;$$

If the resulting state is *safe* $\rightarrow$ the resources are allocated to P_i .

If *unsafe* $\rightarrow P_i$ **must wait**, and the old resource-allocation state is **restored**.

Banker's Algorithm (8/9)

■ Example:

- A system with five processes $\{P_0, P_1, \dots, P_4\}$ and three resource types $\{A, B, C\}$.
- At time T_0 :

<i>Allocation</i>				<i>Max</i>				<i>Available</i>			<i>Need</i>			
	<i>A</i>	<i>B</i>	<i>C</i>		<i>A</i>	<i>B</i>	<i>C</i>	<i>A</i>	<i>B</i>	<i>C</i>		<i>A</i>	<i>B</i>	<i>C</i>
<i>P</i> ₀	0	1	0	<i>P</i> ₀	7	5	3	3	3	2	<i>P</i> ₀	7	4	3
<i>P</i> ₁	2	0	0	<i>P</i> ₁	3	2	2				<i>P</i> ₁	1	2	2
<i>P</i> ₂	3	0	2	<i>P</i> ₂	9	0	2				<i>P</i> ₂	6	0	0
<i>P</i> ₃	2	1	1	<i>P</i> ₃	2	2	2	<i>Finish</i>			<i>P</i> ₃	0	1	1
<i>P</i> ₄	0	0	2	<i>P</i> ₄	4	3	3	<i>P</i> ₀	false		<i>P</i> ₄	4	3	1
								<i>P</i> ₁	false		Safe sequence: < <i>P</i> ₁ <i>P</i> ₃ <i>P</i> ₄ <i>P</i> ₀ <i>P</i> ₂ >			
								<i>P</i> ₂	false					
								<i>P</i> ₃	false					
								<i>P</i> ₄	false					
				<i>Work</i>										
					<i>A</i>	<i>B</i>	<i>C</i>							
					3	3	2							

Banker's Algorithm (9/9)

- At time t_1 , P_1 requests one additional instance of A, two instances of C, so $Request_1 = (1,0,2)$.
- Step 1, $Request_1 \leq Need_1$, $(1,0,2) \leq (1,2,2) \rightarrow \text{ok}$.
- Step 2, $Request_1 \leq Available$, $(1,0,2) \leq (3,3,2) \rightarrow \text{ok}$.
- We **pretend** that this request has been fulfilled, and check the following new state:

<i>Allocation</i>				<i>Need</i>				<i>Available</i>			
	A	B	C		A	B	C	A	B	C	
P_0	0	1	0	P_0	7	4	3	2	3	0	
P_1	3	0	2	P_1	0	2	0				
P_2	3	0	2	P_2	6	0	0				
P_3	2	1	1	P_3	0	1	1				
P_4	0	0	2	P_4	4	3	1				

$\langle P_1, P_3, P_4, P_0, P_2 \rangle$
is a safe sequence

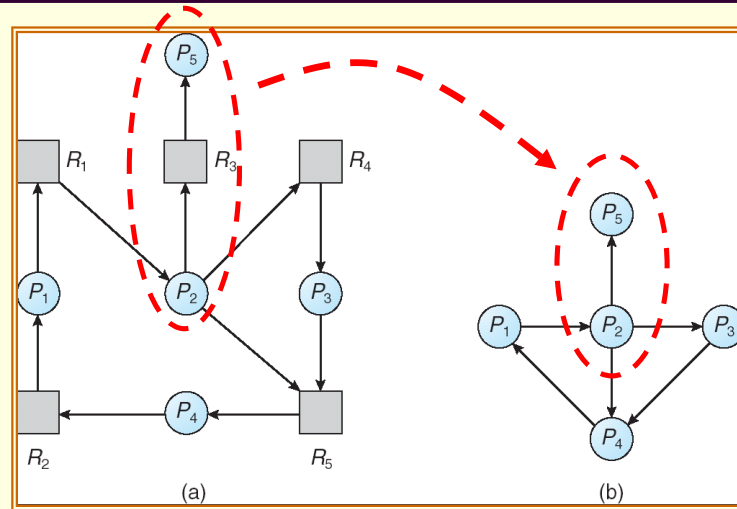
Deadlock Detection

- If a system does not employ either a deadlock-prevention or a deadlock-avoidance algorithm ...
 - **A DEADLOCK SITUATION MAY OCCUR!!**
- Then the system must provide:
 - An algorithm that examines the state of the system to determine (**detect**) whether a deadlock has occurred.
 - An algorithm to **recover** from the deadlock.

Single Instance of Each Resource Type (1/2)

- A deadlock detection algorithm that uses a variant of the resource-allocation graph, called a **wait-for graph**.
 - Remove the resource nodes.
 - Collapse the appropriate edges.
 - An edge from P_i to P_j implies that P_i is waiting for P_j to release **a** resource that P_i needs.
 - The corresponding resource allocation graph contains two edges $P_i \rightarrow R_q$ and $R_q \rightarrow P_j$ for **some** resource R_q .

Single Instance of Each Resource Type (2/2)



- A deadlock exists in the system if and only if the wait-for graph **contains a cycle**.
- To detect deadlocks, the system needs to maintain the wait-for graph and ***periodically*** invoke an algorithm that searches for a cycle in the graph.

Several Instances of a Resource Type (1/4)

- Require several time-varying data structure, similar to those used in the banker's algorithm.
 - **Available:**
 - Vector of length m .
 - If $Available[j] = k$, there are k instances of resource type R_j available.
 - **Allocation:**
 - $n \times m$ matrix.
 - If $Allocation[i, j] = k$ then P_i is currently allocated k instances of R_j .
 - **Request:**
 - $n \times m$ matrix.
 - If $Request[i, j] = k$ then P_i is requesting k more instances of R_j .

Several Instances of a Resource Type (2/4)

1. Let $Work$ and $Finish$ be vectors of length m and n , respectively Initialize:
 - (a) $Work = Available$
 - (b) For $i = 1, 2, \dots, n$, if $Allocation_i \neq 0$, then
 $Finish[i] = false$; otherwise, $Finish[i] = true$.
2. Find an index i such that both
 - (a) $Finish[i] == false$
 - (b) $Request_i \leq Work$If no such i exists, go to step 4.

Several Instances of a Resource Type

(3/4)

3. $Work = Work + Allocation_i$
 $Finish[i] = true$
go to step 2.
4. If $Finish[i] == false$, for some i , $1 \leq i \leq n$, then the system is in deadlock state.
 - Moreover, if $Finish[i] == false$, then P_i is deadlocked.
- An optimistic attitude:
 - We reclaim the resources of P_i (step 3) as soon as $Request_i \leq Work$ (step 2b). → 假設不會要更多
 - We assume that P_i will require no more resources to complete its task.
 - If the assumption is incorrect, we check deadlocks the next time the algorithm is invoked.

Several Instances of a Resource Type (4/4)

■ Example:

- A system with five processes $\{P_0, P_1, \dots, P_4\}$ and three resource types $\{A:7, B:2, C:6\}$.
- At time T_0 :

Allocation			Request			Available					
	A	B	C		A	B	C	A	B	C	
P_0	0	1	0	P_0	0	0	0	0	0	0	
P_1	2	0	0	P_1	2	0	2	Work			
P_2	3	0	3	P_2	0	0	0	A	B	C	Finish
P_3	2	1	1	P_3	1	0	0	0	0	0	P_0 false
P_4	0	0	2	P_4	0	0	2				P_1 false
											P_2 false
sequence: P_0 P_2 P_3 P_1 P_4											P_3 false
Finish[i] == true for all i											P_4 false

Detection-Algorithm Usage

- When should we invoke the detection algorithm?
 - If deadlocks occur frequently, the algorithm should be invoked frequently.
 - Otherwise, the number of processes involved in the deadlock cycle may grow!!
 - In the extreme, we can invoke the deadlock-detection algorithm every time a request for allocation **cannot be granted immediately**.
 - Then we can identify not only the deadlocked processes but also the process that caused the deadlock.
 - However, this will incur a considerable overhead in computation time.
 - An alternative is to invoke the algorithm at less frequent intervals.
 - Once per hour or whenever CPU utilization drops below a certain percent.

Recovery From Deadlock (1/2)

- Two options for breaking a deadlock:

1. **Process Termination:**

- Abort all deadlocked processes.
 - Very expensive – a long-running process have to be recomputed.
- Abort one process at a time until the deadlock cycle is eliminated.
 - Still expensive – a deadlock-detection algorithm must be invoke after each elimination, to determine whether any processes are still deadlocked.
 - Many factors can be considered to determine which process should be terminated.
 - Process priority.
 - Computation time (spent or left)
 - ...

Recovery From Deadlock (2/2)

2. **Resource Preempt:**

- Preempt some resources from processes.
- Give these resources to other processes until the deadlock cycle is broken.
- Issues:
 - Selecting a victim.
 - Which resources and which processes are to be preempted?
 - As in process termination, many factors can be considered.
 - Rollback.
 - We can not continue the preempted process as the process is missing some needed resource.
 - We must roll back the process to some safe state and restart it from that state.
 - One simple and popular method – total rollback.
 - Starvation.
 - Can we guarantee that resources will not always be preempted from the same process?
 - A selected victim can be the next victim, using a pre-defined factor consideration mechanism.

End of Chapter 7

Homework :

Exercises:

Due date: